

Day 5 Mission: Investigate How Software Remembers Its History

Today is not about Git commands.

Today is about a problem that every software engineer eventually faces.

Imagine this situation:

You worked on a project for 7 days.

Everything was working.

Today you made a change.

Now the application is broken.

You don't know what changed.

You don't know when it changed.

You don't know how to go back.

Question:

How would you recover?

This is the problem Git was created to solve.

Your mission today is to discover how software projects remember their history.

Learning Rule

For every concept today:

Do NOT search for definitions first.

Follow this process:

Question

↓

Prediction

↓

Experiment

↓
Observation
↓
Explanation

The goal is understanding.

Not command memorization.

Investigation 1: Why Does History Matter?

Create a file:

notes.txt

Write something inside it.

Now modify it.

Then modify it again.

Then modify it again.

Question:

Can you recover the very first version?

Try.

Observe what happens.

Document the difficulty.

Now ask:

If a project contains 10,000 files, how would a team recover old versions?

Investigation 2: What Does Git Actually Track?

Create a new Git repository.

Before doing anything ask:

What is a repository?

Do not memorize the definition.

Investigate.

Questions:

- What changes after initialization?
- Which new files appear?
- What information is Git storing?

Explore the project directory carefully.

Document discoveries.

Investigation 3: What Is a Commit?

Create a file.

Make a commit.

Modify the file.

Make another commit.

Now ask:

What exactly did Git remember?

Questions:

- Did Git save the whole project?
- Did Git save only changes?
- How can Git reconstruct old versions?

Explore the history.

Observe.

Document conclusions.

Investigation 4: Can Time Move Backwards?

After creating multiple commits:

Investigate:

- Viewing old versions
- Returning to previous states
- Comparing different versions

Questions:

How does Git know what existed yesterday?

How does Git recreate old project states?

This is one of the most important discoveries today.

Investigation 5: Why Do Branches Exist?

Imagine two developers.

Both want to modify the same project.

Questions:

Should they work on the same version?

What risks appear?

Create a branch.

Make changes.

Return to the original branch.

Observe:

- What changed?
- What stayed unchanged?

Document observations.

Investigation 6: What Happens During a Merge?

Eventually separate work must come together.

Investigate:

- Merging a branch

- Viewing the result

Questions:

How does Git know what to combine?

What happens if two people change the same line?

Try creating this situation intentionally.

Observe the outcome.

Real understanding appears when things break.

Investigation 7: Break Git

Create confusion intentionally.

Examples:

- Create multiple commits
- Create branches
- Make conflicting changes

Then try to understand:

- What happened?
- Why?
- How can you recover?

Engineers learn Git fastest through recovery.

Not success.

Investigation 8: What Problem Does Git Solve?

By now you have used:

- Repository
- Commit
- Branch
- Merge

Forget the commands.

Answer:

What problem does each concept solve?

Focus on the problem.

Not the feature.

Deliverables

1. Git Investigation Repository

Create a repository used for today's experiments.

Include:

- Multiple commits
 - Multiple branches
 - At least one merge
-

2. Discovery Notes

Create:

day5-discoveries.md

For every investigation record:

- Prediction
 - Experiment
 - Observation
 - Conclusion
-

3. Evidence

Take screenshots showing:

- Repository creation
 - Commit history
 - Branches
 - Merge results
 - Any errors encountered
-

Success Criteria

You succeed today if:

- You understand why Git exists.
- You can recover previous versions confidently.
- You understand why branches are necessary.
- You understand why merges exist.
- Git feels like a solution to real engineering problems rather than a list of commands.

If you only learned commands, repeat the investigations.

If you understand the problems Git solves, you have completed the mission.